

Chapter 4B: Adversarial Search and Games (Part 2)

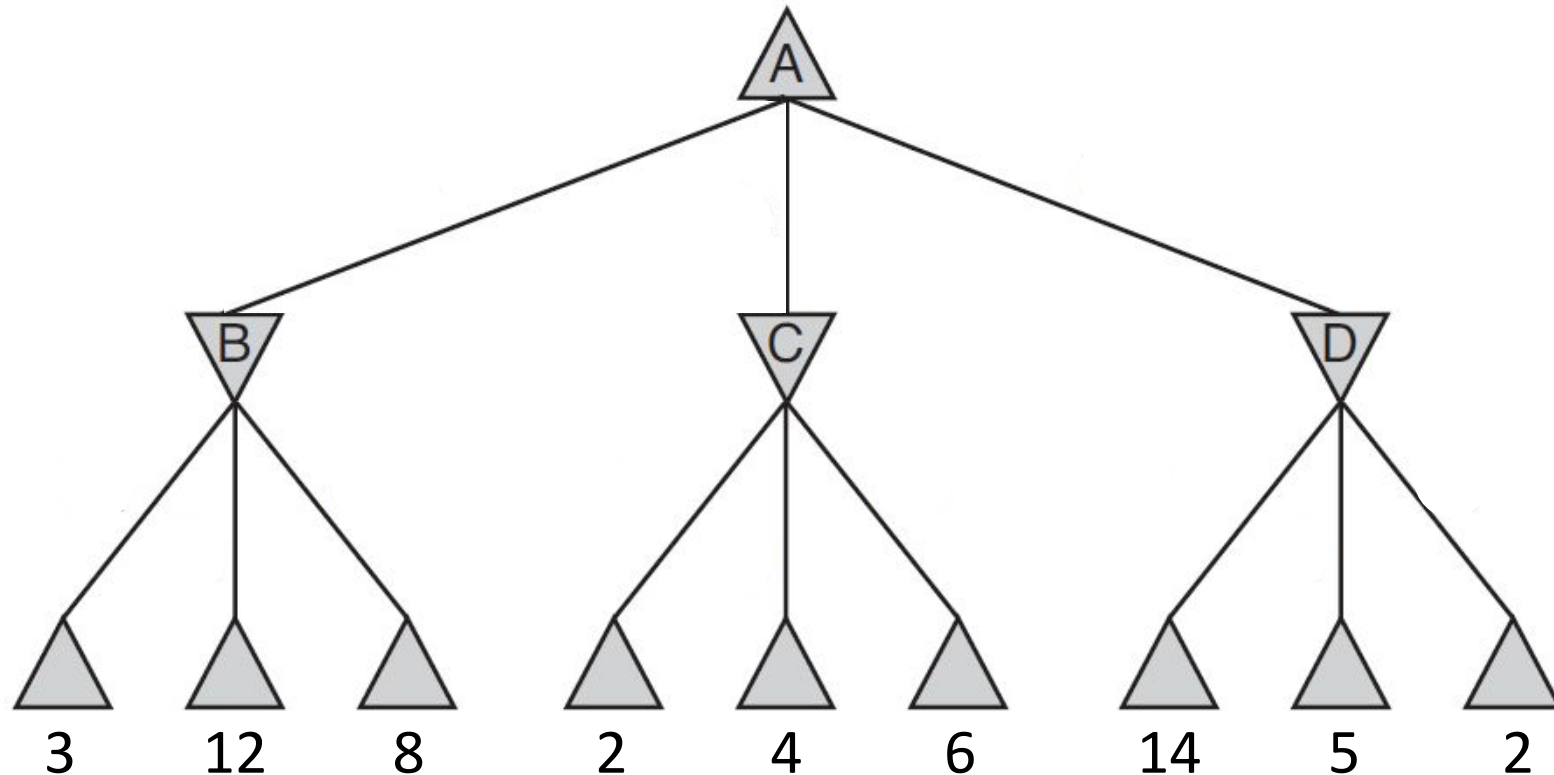
AMCS2104 Fundamentals of Artificial Intelligence

Content

- Limitation of Alpha-Beta Pruning
- Move Ordering
- Transposition
- Type A Strategy vs. Type B Strategy
- Heuristic Evaluation Function
- Cut-Off Search
- Forward Pruning
- PROBCUT Algorithm
- Search vs. Lookup
- Improved minimax Search

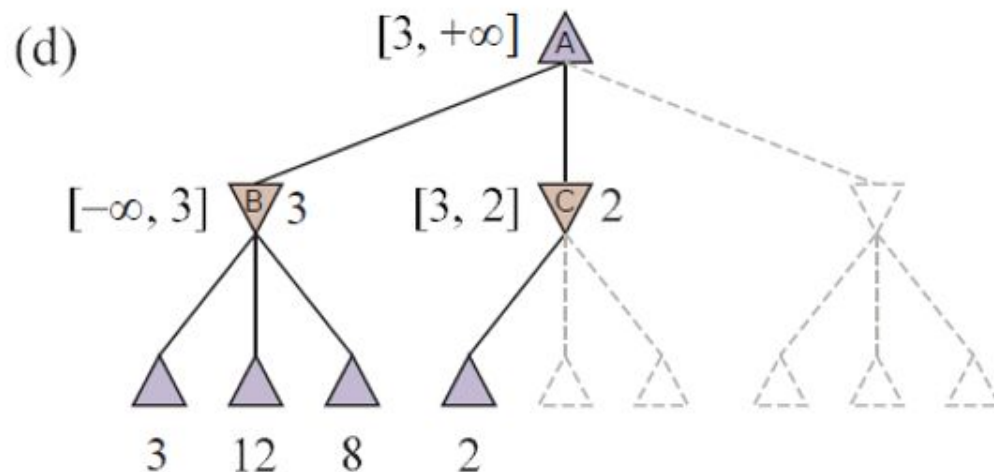
Limitation of Alpha-Beta Pruning

- Apply minimax algorithm to the game tree with alpha-beta pruning in **right-to-left order**. Then, identify nodes to be pruned.



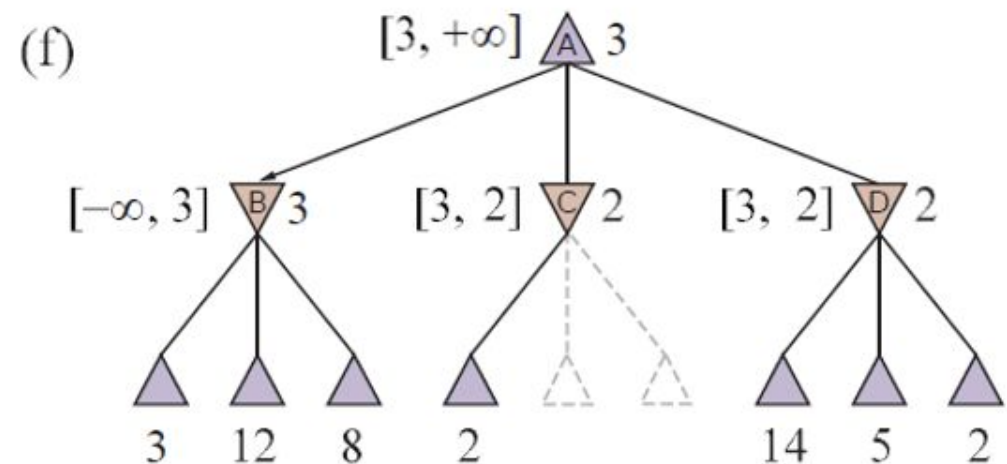
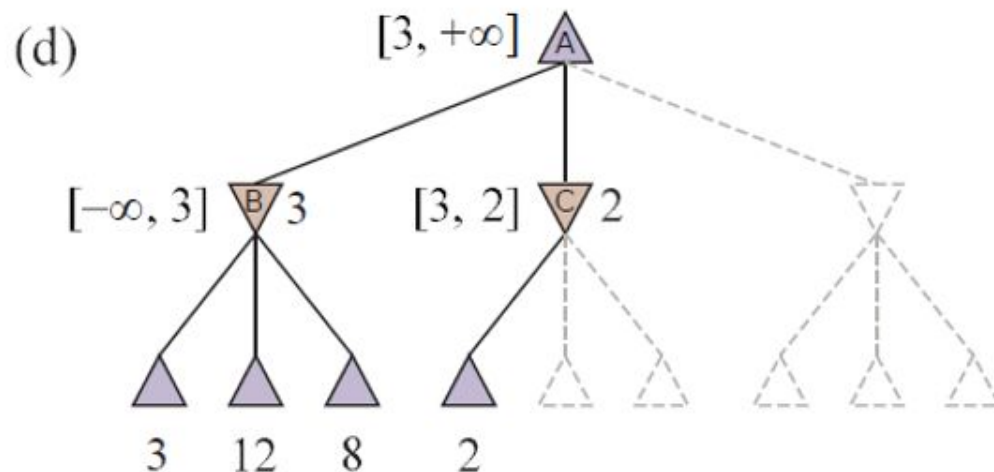
Limitation of Alpha-Beta Pruning

- The effectiveness of alpha-beta pruning is highly dependent on the **order** in which the states are examined.
- In Step (d), pruning can happen because the **best successor** (from the point of view of MIN) is **generated first**.



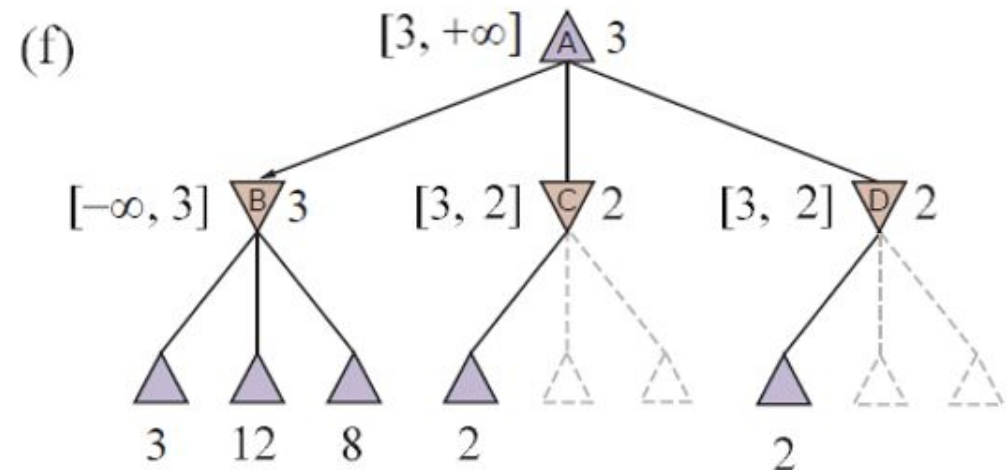
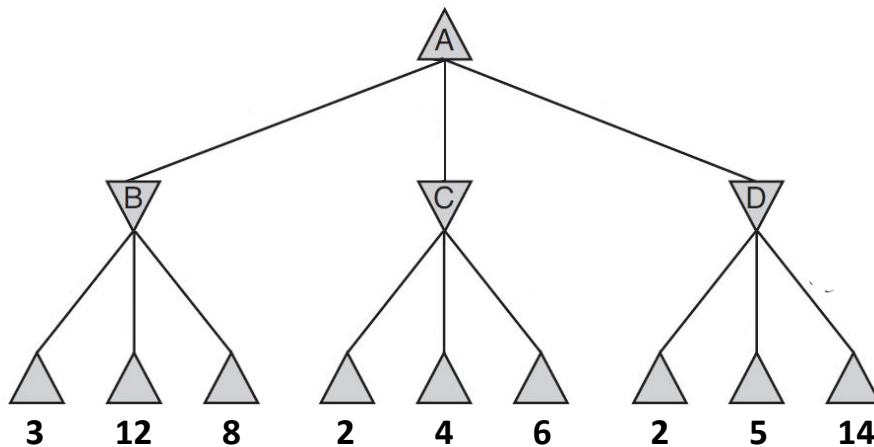
Limitation of Alpha-Beta Pruning

- The effectiveness of alpha-beta pruning is highly dependent on the **order** in which the states are examined.
- In Step (f), pruning cannot happen because the **worst successor** (from the point of view of MIN) is **generated first**.



Limitation of Alpha-Beta Pruning

- The effectiveness of alpha-beta pruning is highly dependent on the **order** in which the states are examined.
- In Step (f), pruning can happen if the **best successor** (from the point of view of MIN) is **generated first**.



Limitation of Alpha-Beta Pruning

- Thus, it is worthwhile to examine first the **successors** that are likely to be **best**.
 - Depth-first move
 - Nodes to be examined: $O(b^m)$.
 - The best move
 - Also called **killer move**.
 - Nodes to be examined: $O(b^{m/2})$.
 - The **effective branching factor** will become $\sqrt{b}$ instead of b .
 - Example: Chess' branching factor will be reduced from 35 to about 6.
- Random move
 - Nodes to be examined: $O(b^{3m/4})$.

Move Ordering

- Addition of **dynamic move-ordering schemes** to try the best move found in the past:
 - From the previous move.
 - From the previous exploration of the current move.
 - Implementation: **Iterative deepening search**
 - First, search one ply deep and record the best path of moves.
 - Then search one ply deeper, but use the recorded path to inform move ordering.

Transpositions

- In many games, repeated states occur frequently because of **transpositions**.
 - Different permutations of a move sequence ends up in the same position.

1. e4	e5
2. Nf3	Nc6
1. Nf3	Nc6
2. e4	e5
1. e4	Nc6
2. Nf3	e5



Transpositions

- ❑ **Redundant paths** to repeated states can cause an exponential increase in search cost.
- ❑ However, keeping a **transposition table** of previously reached states can address this problem.
 - ❑ A **transposition table** is a cache of previously seen positions.
- ❑ In chess, the use of transposition tables is very effective, allowing us to **double the reachable search depth in the same amount of time**.
- ❑ However, keeping previously reached states in a transposition table could be an issue if the space complexity is exponential.

Transpositions

□ Example: Chess

- Suppose **White** has a move w_1 that can be answered by **Black** with b_1 and an unrelated move w_2 on the other side of the board that can be answered by b_2 , and that we search the sequence of moves $[w_1, b_1, w_2, b_2]$ and let's call the resulting state s .
- After exploring a large sub-tree below s , we find its **backed-up value**, which we store in the transposition table.
- When we later search the sequence of moves $[w_2, b_2, w_1, b_1]$, we end up in s again, and we can **look up the value instead of repeating the search**.

minimax Search with Alpha-Beta Pruning

□ Problem

- Minimax algorithm generates the entire game search space.

□ Solution

- Alpha-beta algorithm allows us to **prune large parts of it**.
- A **clever move ordering** can help maximize the effectiveness of pruning.

□ Problem

- When the depth of a game tree is **impractically deep**, the above-mentioned solution fails as well.

Type A Strategy vs. Type B Strategy

- Claude Shannon's paper **Programming a Computer for Playing Chess** (1950):
 - **Type A Strategy**
 - Explores a **wide but shallow** portion of the tree.
 - Considers all possible moves to a **certain depth** in the search tree.
 - Uses a **heuristic evaluation function** to estimate the utility of states at that depth.
 - **Type B Strategy**
 - Explores a **deep but narrow** portion of the tree.
 - Ignores moves that look bad.
 - Follows promising lines "**as far as possible**".

Heuristic Alpha-Beta Tree Search

- Amendment on minimax search with alpha-beta pruning:
 - Replace the **utility function** with a **heuristic evaluation function** $\text{EVAL}(s, p)$, which estimates the position's utility.
 - Replace the **terminal test** with a **cutoff test**.

$$\text{H-MINIMAX}(s, d) = \begin{cases} \text{EVAL}(s, \text{MAX}) & \text{if IS-CUTOFF}(s, d) \\ \max_{a \in \text{Actions}(s)} \text{H-MINIMAX}(\text{RESULT}(s, a), d + 1) & \text{if TO-MOVE}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{H-MINIMAX}(\text{RESULT}(s, a), d + 1) & \text{if TO-MOVE}(s) = \text{MIN}. \end{cases}$$

where $\text{H-MINIMAX}(s, d)$ is the **heuristic minimax value** for state s and depth d .

Designing Good Heuristic Evaluation Functions

□ Time

- The computation must not take too long.
- The whole point is to search faster.

□ Performance

- For nonterminal states, the evaluation function should be **strongly correlated with the actual chances of winning**.
- Most evaluation functions work by calculating various features of the state.

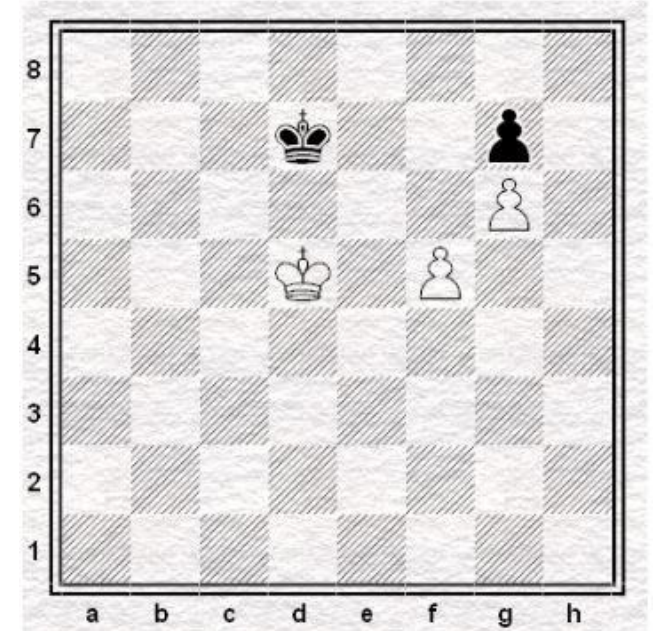
Heuristic Design Approach #1

- Example: Chess
 - **Features:** The number of white pawns, black pawns, white queens, black queens, and so on.
 - Different combinations of features define **various categories of states**.
 - Any of these categories contain:
 - Some states that lead to **wins**.
 - Some that lead to **draws**.
 - Some that lead to **losses**.
 - Example
 - One category might contain all two-pawn versus one-pawn endgames.

Heuristic Design Approach #1

- Example: Two-pawn vs. one-pawn endgames category
 - Suppose our experience suggests that:
 - 82% lead to a win (utility +1)
 - 2% lead to a loss (utility 0)
 - 16% lead to a draw (utility $\frac{1}{2}$)
 - A reasonable evaluation for states in the category is the expected value:

$$(0.82 \times +1) + (0.02 \times 0) + (0.16 \times \frac{1}{2}) = 0.90$$



Heuristic Design Approach #1

- This approach is not practical:
 - It requires **too many categories**.
 - It requires **too much experience** to estimate all the probabilities.

Heuristic Design Approach #2

- Evaluation functions **compute separate numerical contributions from each feature** and then **combine them to find the total value**.
- Example: Features
 - Approximate material value (Introductory chess books)
 - A pawn is worth 1.
 - A knight or bishop is worth 3.
 - A rook is worth 5.
 - A queen is worth 9.
 - Other features
 - Good pawn structure and King safety are worth half a pawn.

Heuristic Design Approach #2

- These feature values are then simply added up to obtain the evaluation of the position.
- This kind of evaluation function is called a **weighted linear function**:

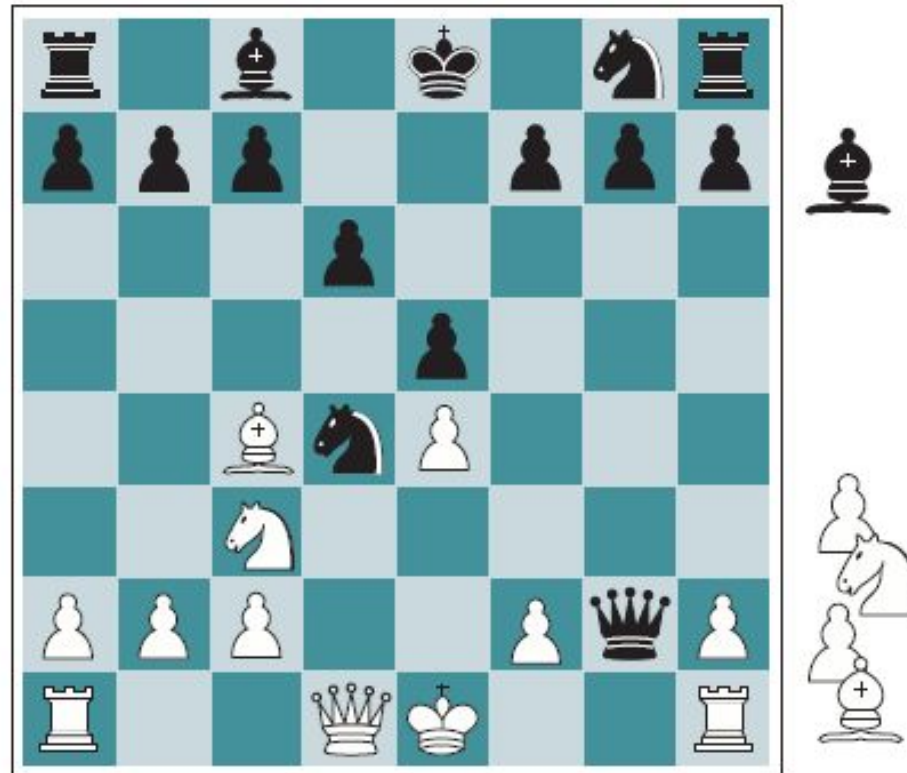
$$\text{EVAL}(s) = w_1 f_1(s) + w_2 f_2(s) + \cdots + w_n f_n(s) = \sum_{i=1}^n w_i f_i(s)$$

where each f_i is a feature of the position (such as "number of white bishops"), and w_i is a weight (saying how important that feature is)

- The weights are normalized so that the sum is always within the range of a loss (0) to a win (+1).

Heuristic Design Approach #2

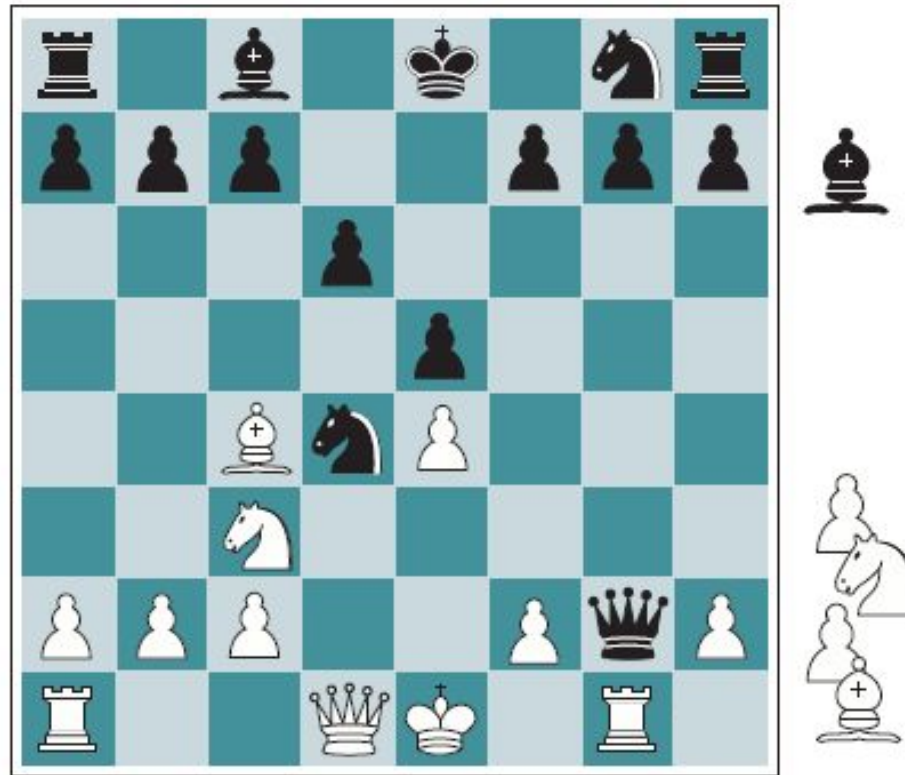
- Black has an advantage of a knight and two pawns, which should be enough to win the game.



(a) White to move

Heuristic Design Approach #2: Issues

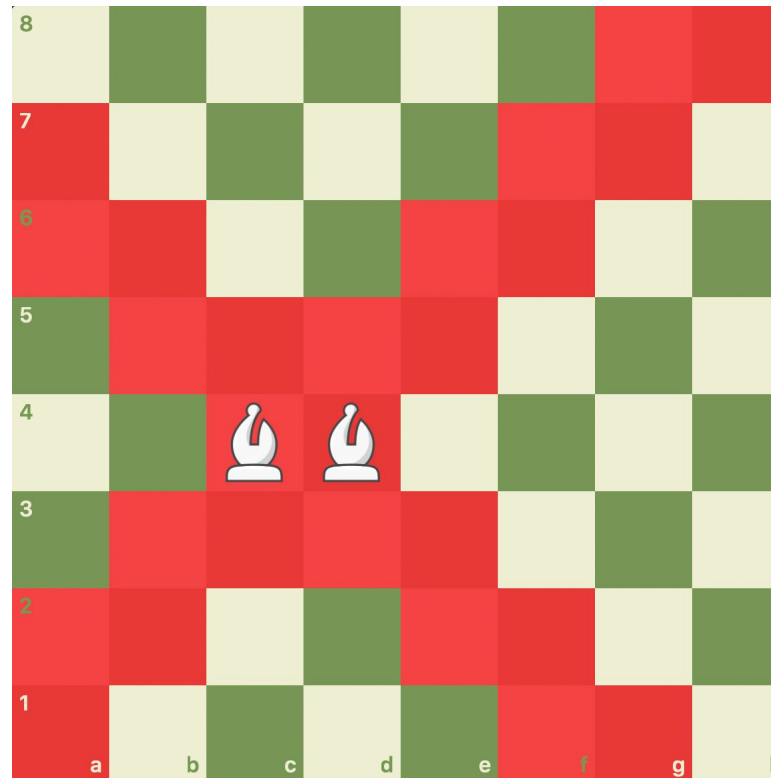
- Does Black still have the advantage of a knight and two pawns?



(b) White to move

Heuristic Design Approach #2: Issues

- The **weighted linear function** is used on top of a strong assumption in which the contribution of each feature is **independent** of the values of the other features.

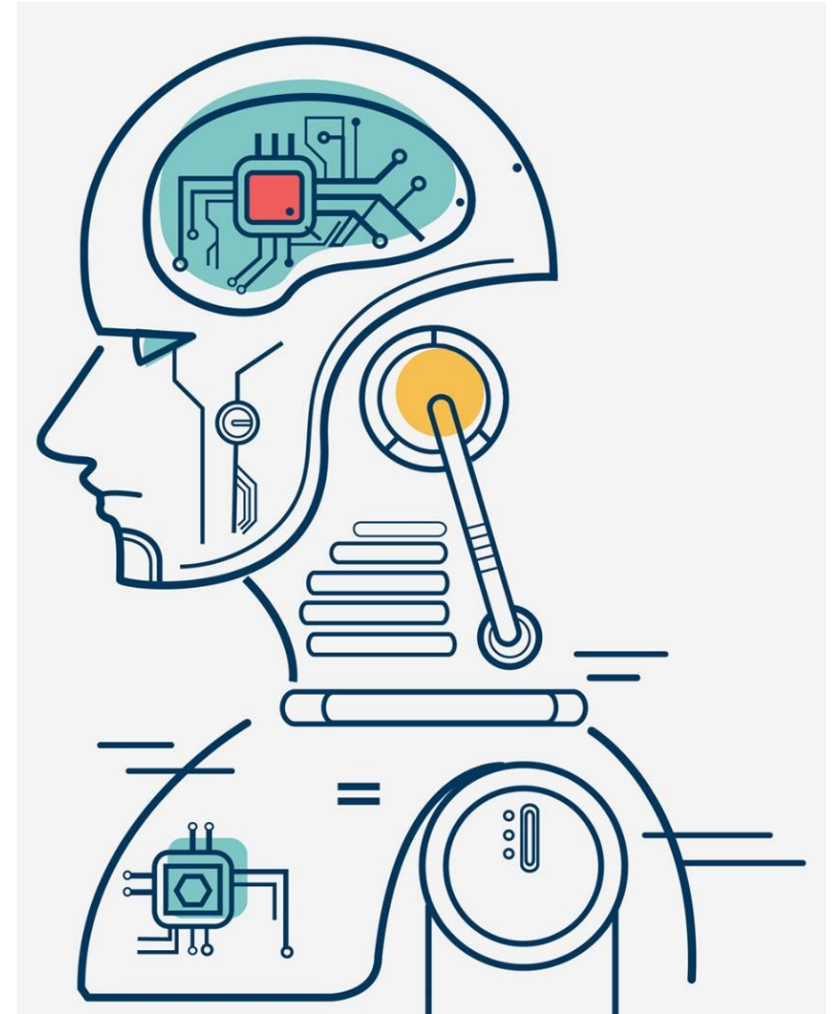


Heuristic Design Approach #2: Solutions

- Example
 - A pair of bishops might be worth more than twice the value of a single bishop.
 - A bishop is worth more in the endgame than earlier when:
 - The move number feature is high, or
 - The number of remaining pieces feature is low.
- As a result, current programs for chess and other games also use **non-linear combinations** of features.
- **Question:** How about the **features** and **weights**? Where do they come from?

Heuristic Design Approach #2: Solutions

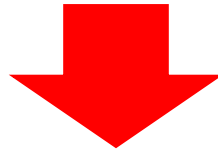
- The **features** and **weights** come from centuries of **human chess-playing experience**.
- In games where this kind of experience is not available, the weights of the evaluation function can be estimated by the **machine learning** techniques.
- Example
 - Applying these techniques to chess has confirmed that a bishop is indeed worth about three pawns.



Cutting Off Search

- **Cut-off search** can be implemented by replacing the two lines in the **alpha-beta search algorithm** that mention **IS-TERMINAL** with the following line:

if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null



if game.IS-CUTOFF(state, depth) then return game.EVAL(state, player), null

```
function ALPHA-BETA-SEARCH(game, state) returns an action  
  player  $\leftarrow$  game.TO-MOVE(state)  
  value, move  $\leftarrow$  MAX-VALUE(game, state,  $-\infty$ ,  $+\infty$ )  
  return move
```



```
function MAX-VALUE(game, state,  $\alpha$ ,  $\beta$ ) returns a (utility, move) pair  
  if game.IS-CUTOFF(state, depth) then return game.EVAL(state, player), null  
  v  $\leftarrow -\infty$   
  for each a in game.ACTIONS(state) do  
    v2, a2  $\leftarrow$  MIN-VALUE(game, game.RESULT(state, a),  $\alpha$ ,  $\beta$ )  
    if v2 > v then  
      v, move  $\leftarrow$  v2, a  
       $\alpha \leftarrow$  MAX( $\alpha$ , v)  
    if v  $\geq$   $\beta$  then return v, move  
  return v, move
```



```
function MIN-VALUE(game, state,  $\alpha$ ,  $\beta$ ) returns a (utility, move) pair  
  if game.IS-CUTOFF(state, depth) then return game.EVAL(state, player), null  
  v  $\leftarrow +\infty$   
  for each a in game.ACTIONS(state) do  
    v2, a2  $\leftarrow$  MAX-VALUE(game, game.RESULT(state, a),  $\alpha$ ,  $\beta$ )  
    if v2 < v then  
      v, move  $\leftarrow$  v2, a  
       $\beta \leftarrow$  MIN( $\beta$ , v)  
    if v  $\leq$   $\alpha$  then return v, move  
  return v, move
```

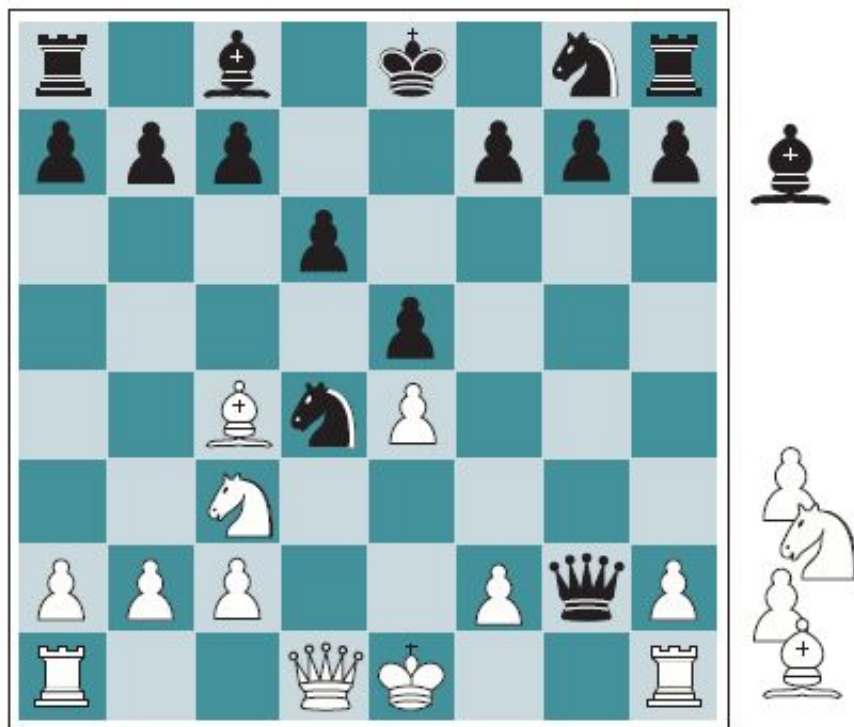
Determining the Cut-Off Point

- Some **bookkeeping** has to be arranged so that the **current depth** can be incremented on each recursive call.
- **Approaches**
 - **Set a fixed depth limit** so that **IS-CUTOFF(state, depth)** returns true for all depth greater than some fixed depth *d*.
 - The depth *d* is properly chosen so that a move is selected within the allocated time.
 - Alternatively, **iterative deepening** is applied and when time runs out, the program returns the move selected by the deepest completed search.

Limitation of Cut-Off Search

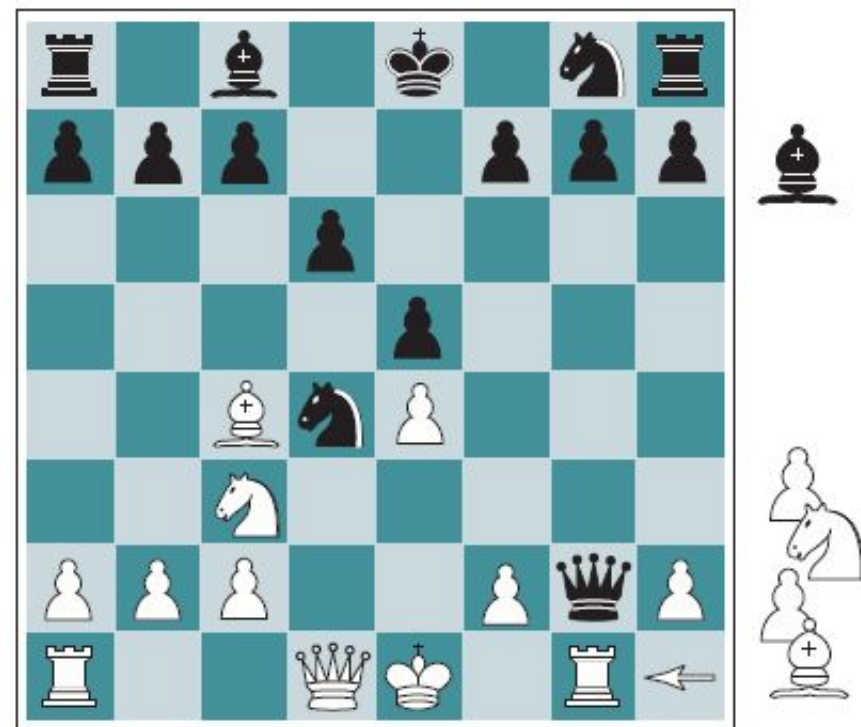
- Quiescent positions vs. **non-quiescent positions**:

Quiescent position



(a) White to move

Non-quiescent position



(b) White to move

Limitation of Cut-Off Search

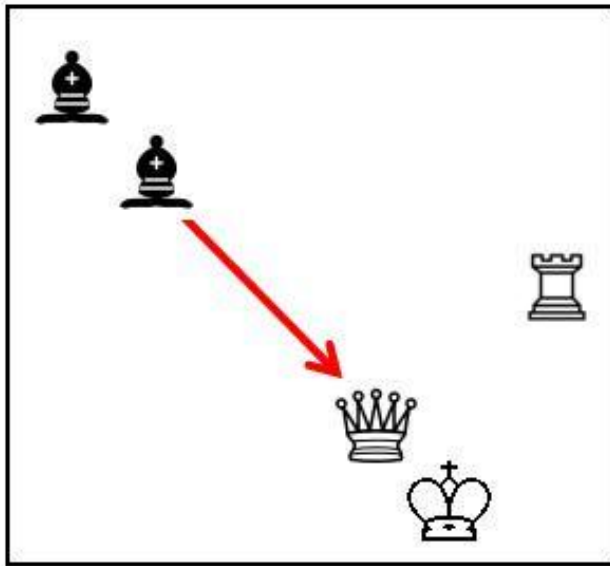
- Suppose the program searches to the depth limit, reaching the position in Figure (b) in the previous slide, where Black is ahead by a knight and two pawns.
- It would report this as the heuristic value of the state, thereby declaring that the state is a probable win by Black.
- But White's next move captures Black's queen with no compensation.
- Hence, the position is actually favorable for White, but this can be seen only by looking ahead.

Limitation of Cut-Off Search

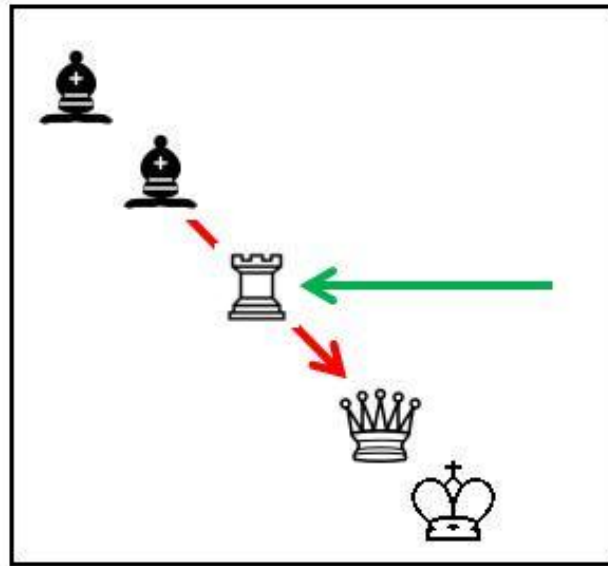
- Cutting-off search fails to handle **non-quiet positions**.
- Thus, a more sophisticated cutoff test is needed:
 - **Quiet positions**
 - The evaluation function should be applied.
 - **Non-quiet positions**
 - Extra search called **quiescence search** is conducted until quiet positions are reached.

Limitation of Cut-Off Search

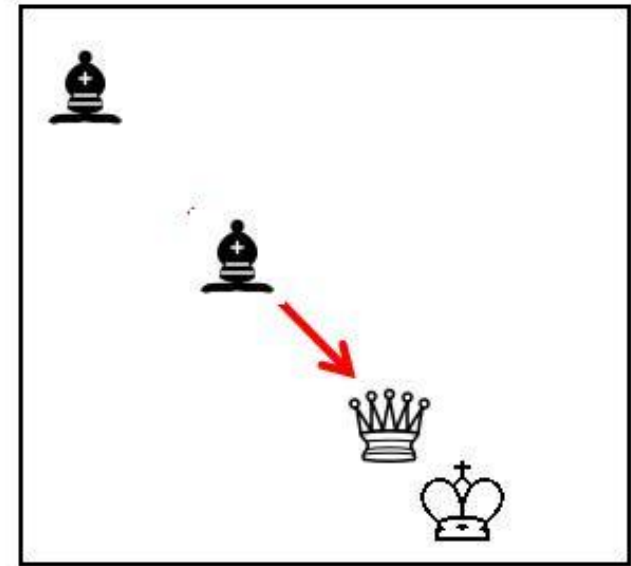
- A **horizon effect** arises when the program is facing an opponent's move that causes serious and unavoidable damage and is **ultimately unavoidable**, but can be temporarily avoided by delaying tactics.



Queen pinned by bishops



Could move rook in way



But rook captured and will still lose queen

Limitation of Cut-Off Search

- Cut-off search also fails to handle **horizon effect**.
- One strategy to reduce the horizon effect is the **singular extensions**.
- **Singular extensions** refer to "clearly better" moves compared with all other moves in a given position, even when the search would normally be cut off at that point:
 - Once discovered anywhere in the tree in the course of a search, this singular move is remembered.
 - When the search reaches the normal depth limit, the algorithm checks to see if the singular extension is a legal move.
 - If it is, the algorithm allows the move to be considered.

Forward Pruning

- **Forward pruning** is a Type B strategy.
- **Forward pruning** cuts away moves that appear to be poor moves.
- It **saves computation time** at the risk of **making an error**.
- One approach to forward pruning is **beam search**:
 - On each ply, consider only a "beam" of the ***n*** best moves (according to the evaluation function) rather than considering all possible moves.
 - But this approach is too risky as there is no guarantee that the best move will not be pruned away.

PROBCUT Algorithm

- The **PROBCUT** algorithm
 - Stands for probabilistic cut, which means pruned by probability.
 - Proposed by Michael Buro in 1995
 - A **forward-pruning version of alpha-beta search**.
 - It uses statistics gained from prior experience to lessen the chance that the best move will be pruned
- When **PROBCUT** is applied to an Othello program, it outperforms the regular version 64% of the time, even when the regular version was given twice as much time.

PROBCUT Algorithm

- PROBCUT has been proven to be effective in Othello and Shogi.



9	8	7	6	5	4	3	2	1	
香	桂	銀	金	王	金	銀	桂	香	a
	飛						角		b
歩	歩	歩	歩	歩	歩	歩	歩	歩	c
									d
									e
									f
歩	歩	歩	歩	歩	歩	歩	歩	歩	g
	角						飛		h
香	桂	銀	金	玉	金	銀	桂	香	i

Search vs. Lookup

- ❑ **Question:** What is the best first move?
- ❑ It doesn't make sense for a chess program to consider a tree of a billion game states for the best first move.
- ❑ Plenty of books describing good play in the opening and endgame in chess are available
- ❑ **Question:** How can this help solving the problem?



Search vs. Lookup

- Many game-playing programs use table lookup rather than search for the opening and ending of games.
- A switch between search and table lookup:
 - Opening (fewer choices)
 - **Table lookup:** Humans expertise + the winning statistics from a database.
 - After approximate 10 to 15 moves (more choices)
 - **Search algorithm**
 - Ending (fewer choices)
 - **Table lookup:** Humans expertise + the winning statistics from a database

Improved minimax Search

- minimax search with
 - + a reasonable cutoff test with a quiescence search
 - Look ahead 5 ply ==> Average level
 - Can be defeated by an average human chess player (i.e., 6 - 8 ply)
 - + alpha-beta pruning with a large transposition table
 - Look ahead 14 ply ==> Expert level
 - + Extensively tuned evaluation function and a large DB of endgame moves
 - Look ahead ≥ 30 ply ==> Grandmaster level

Reading Homework

- **Artificial Intelligence: A Modern Approach**, 4th Edition, by Stuart Russell and Peter Norvig
 - Chapter 6: Adversarial Search (page 199 to page 207)

The End